



Universidad
Continental

ASIGNATURA: ANÁLISIS Y DISEÑO DE
SOFTWARE

DESARROLLO DE UN SISTEMA PARA GESTIÓN DE INVENTARIOS EN COMEDORES COMUNITARIOS

DOCENTE: ROSARIO OSORIO CONTRERAS

INTEGRANTES:

- DE LA CRUZ HUAMANI, JHAYR AXCEL
- ALMERCO TICSE PIERO ALDAIR



ODS 2: HAMBRE CERO- COMEDORES

Problema identificado :

Los comedores comunitarios presentan problemas en el control de sus recursos alimentarios debido al uso de procesos manuales y desorganizados, generando pérdidas de alimentos por vencimiento y dificultades en el manejo del inventario. Además, la falta de información actualizada sobre productos disponibles y próximos a vencer dificulta priorizar adecuadamente los alimentos durante la preparación de raciones. Por ello, se propone desarrollar un sistema web orientado al control de inventarios alimentarios, incorporando alertas de vencimiento, control de stock y la lógica FEFO para reducir el desperdicio de alimentos.



SOLUCIÓN PROPUESTA:



Se propone el desarrollo de un sistema de gestión de inventario orientado a comedores comunitarios, el cual permitirá registrar, controlar y monitorear los insumos alimentarios desde su ingreso hasta su consumo. El sistema facilitará el registro de productos, lotes, cantidades disponibles y fechas de vencimiento, así como el seguimiento de los movimientos de entrada y salida de alimentos. La solución incluirá funcionalidades para la visualización del estado actual del inventario, permitiendo identificar de manera rápida los productos disponibles, aquellos con bajo stock y los próximos a vencer mediante alertas automáticas de vencimiento y stock crítico. Esto permitirá a los responsables del comedor tomar decisiones informadas sobre el uso prioritario de los alimentos. Asimismo, el sistema contará con un historial de movimientos que brindará trazabilidad sobre el uso de los insumos, facilitando el control interno y la transparencia en la gestión de los recursos. Además, se generarán reportes básicos que permitirán analizar el consumo de alimentos y el estado del inventario en periodos determinados. El sistema estará enfocado principalmente en la gestión de inventarios alimentarios; sin embargo, también brindará apoyo básico a la gestión alimentaria mediante consultas de disponibilidad de insumos y sugerencias de consumo prioritario basadas en la lógica FEFO.

CAPÍTULO 2. ANÁLISIS DE NECESIDADES Y REQUERIMIENTOS

El problema radica en la opacidad de los datos. Al usar cuadernos o procesos manuales, la información sobre el estado de los alimentos es "estática" y no genera alertas. Esto produce tres efectos negativos:

- Vencimiento invisible:** Los alimentos con fechas próximas de caducidad quedan enterrados bajo nuevas donaciones.
- Incertidumbre operativa:** El encargado de cocina no sabe qué menús puede planificar a futuro porque no tiene un conteo exacto de insumos.
- Desperdicio económico:** Cada producto vencido es una pérdida de inversión o de donación que impacta directamente en la cantidad de raciones diarias.

Usuario	Necesidad Principal
Encargado de Almacén	Registrar entradas y salidas de productos de forma rápida, así como visualizar alertas de productos próximos a vencer para evitar su pérdida.
Administrador del Comedor	Conocer el stock actual para solicitar requerimientos con base en faltantes reales y generar reportes de transparencia.
Cocinero/a	Consulta disponibilidad de productos, alertas de vencimiento y sugerencias FEFO para preparación de raciones.

REQUERIMIENTO FUNCIONALES (RF):

RF1 (Gestión de Entrada): El sistema debe permitir registrar lotes de productos incluyendo: producto, cantidad , proveedor, fecha de ingreso, fecha de vencimiento, unidad de medida

RF02 – Actualización de productos: El sistema deberá permitir editar la información de los productos registrados (cantidad, fecha de vencimiento, categoría).

RF03 – Registro de entradas de insumos: El sistema debe permitir registrar el ingreso de productos, aumentando automáticamente el stock.

RF04– registro de salida de insumos:El sistema debe registrar automáticamente el lote específico utilizado durante cada salida de inventario aplicando la lógica FEFO.

RF05 – Visualización de inventario: El sistema deberá mostrar el estado actual del inventario, incluyendo productos disponibles, cantidades y fechas de vencimiento.

RF06 – Alertas de productos próximos a vencer: El sistema generará alertas automáticas cuando falten 7 días para el vencimiento de un lote.

RF07 – Historial de movimientos: El sistema deberá almacenar y permitir consultar el historial de entradas y salidas de cada producto.

RF08 – Generación de reportes: El sistema deberá generar reportes básicos sobre consumo de alimentos e inventario cada semana

RF9 – Gestión de usuarios: El sistema deberá permitir el registro y autenticación de usuarios con distintos roles (administrador, encargado del almacén, cocinero).

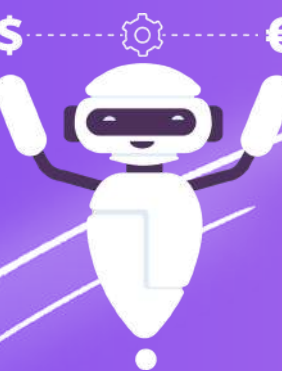
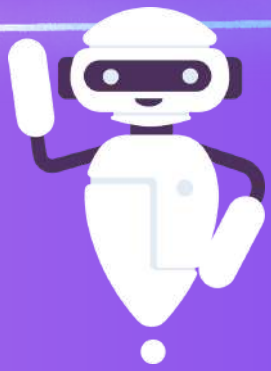
RF10 – Gestión de lotes : El sistema deberá administrar múltiples lotes por producto permitiendo identificar cantidades y fechas de vencimiento independientes.

RF10 – Stock crítico: El sistema deberá generar alertas automáticas cuando la cantidad disponible de un producto o lote se encuentre por debajo del stock mínimo establecido.

RF10 – Desperdicio evitado : El sistema deberá calcular alimentos aprovechados antes de su vencimiento mediante la lógica FEFO.

RF10 – Indicadores ODS 2 : El sistema deberá mostrar métricas relacionadas con desperdicio evitado, productos vencidos y alimentos aprovechados.

RF10 – Aplicar lógica FEFO: El sistema deberá priorizar automáticamente la salida de lotes con fecha de vencimiento más próxima.



REQUERIMIENTO NO FUNCIONALES (RNF):

RNF01 – Usabilidad: El sistema debe presentar una interfaz simple e intuitiva, permitiendo que un usuario registre un producto en un tiempo máximo de 30 segundos, utilizando como máximo 3 pasos (ingreso de datos y confirmación).

RNF02 – Disponibilidad: El sistema debe estar disponible al menos el 99% del tiempo durante el horario de uso (6:00 am – 8:00 pm) y cargar cada vista en un tiempo máximo de 3 segundos en redes 3G o 4G.

RNF03 – Seguridad: El sistema debe implementar autenticación mediante usuario y contraseña, diferenciando roles de usuario (Administrador, Encargado y Cocinero). El Administrador podrá gestionar usuarios, inventario y reportes; el Encargado registrará entradas y salidas; mientras que el Cocinero podrá consultar disponibilidad de productos y alertas de vencimiento.

RNF04 – Rendimiento: El sistema debe procesar operaciones (registro, consulta) en un tiempo máximo de 2 segundos.

RNF05 – Integridad de datos: El sistema debe validar que las cantidades ingresadas sean mayores a 0 y que no se registren productos duplicados

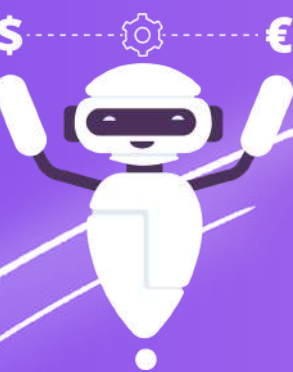
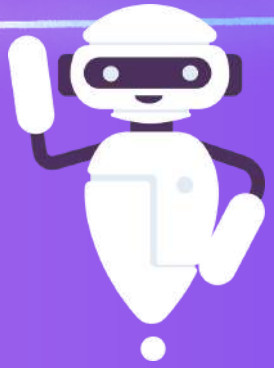
RNF06 – Seguridad y autenticación: El sistema deberá implementar autenticación mediante JWT y control de acceso basado en roles (RBAC) para restringir funcionalidades según el perfil del usuario.

RNF07 – Protección de información: El sistema deberá garantizar sesiones seguras y proteger la información mediante validaciones de acceso y control de permisos.

RNF08 – Disponibilidad: El sistema deberá permitir el registro continuo de movimientos de inventario durante el horario operativo del comedor comunitario.

RNF09 – Escalabilidad :La arquitectura del sistema deberá permitir futuras ampliaciones relacionadas con reportes, indicadores ODS y módulos complementarios de gestión alimentaria.

RNF010 – Integridad de registros: Los movimientos de inventario no podrán eliminarse físicamente y deberán conservar trazabilidad histórica mediante mecanismos de auditoría.



REQUERIMIENTO DE DOMINO(RD):

RD01 – Identificación única de productos: No se deben registrar productos duplicados en el inventario.

RD02 – Control de stock no negativo: La cantidad disponible de un producto nunca puede ser menor a cero.

RD03 – Registro obligatorio de fecha de vencimiento: Todo lote registrado en el sistema deberá contar obligatoriamente con una fecha de vencimiento válida.

RD04 – Aplicación de lógica FEFO: Las salidas de productos deberán priorizar automáticamente los lotes con fecha de vencimiento más próxima para reducir pérdidas de alimentos.

RD05 – Restricción de salida por stock: No se puede registrar la salida de un producto si la cantidad solicitada supera el stock disponible.

RD06 – Clasificación de productos: Todo producto debe pertenecer a una categoría definida (por ejemplo: granos, lácteos, verduras, etc.).

RD07 – Estado de productos por vencimiento: Los productos deben clasificarse como: vigentes, próximos a vencer o vencidos.

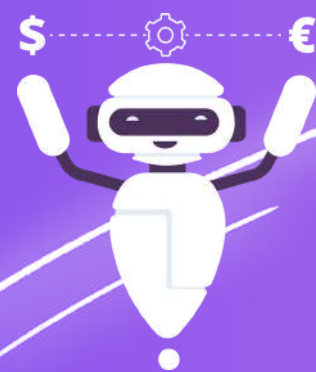
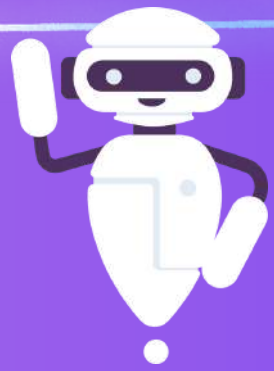
RD08 – Restricción de modificación de historial: Los registros históricos de movimientos de inventario deberán mantenerse íntegros y no podrán ser eliminados ni modificados una vez almacenados.

RD09 – Auditoría de movimientos: Todo movimiento deberá almacenar: Usuario responsable, lote utilizado, fecha, motivo, observaciones. Los registros no podrán eliminarse físicamente.

RD10 – Gestión de lotes : Cada ingreso de productos deberá registrarse mediante lotes independientes con información de cantidad, fecha de ingreso y fecha de vencimiento.

RD11 – Generación de alertas : El sistema deberá generar alertas automáticas cuando un lote se encuentre próximo a vencer dentro del periodo establecido de siete días o cuando el stock sea inferior al mínimo permitido.

RD12 – Conversión de unidades de medida : El sistema deberá permitir la gestión y conversión de unidades de medida para mantener consistencia entre registros de productos e inventario.



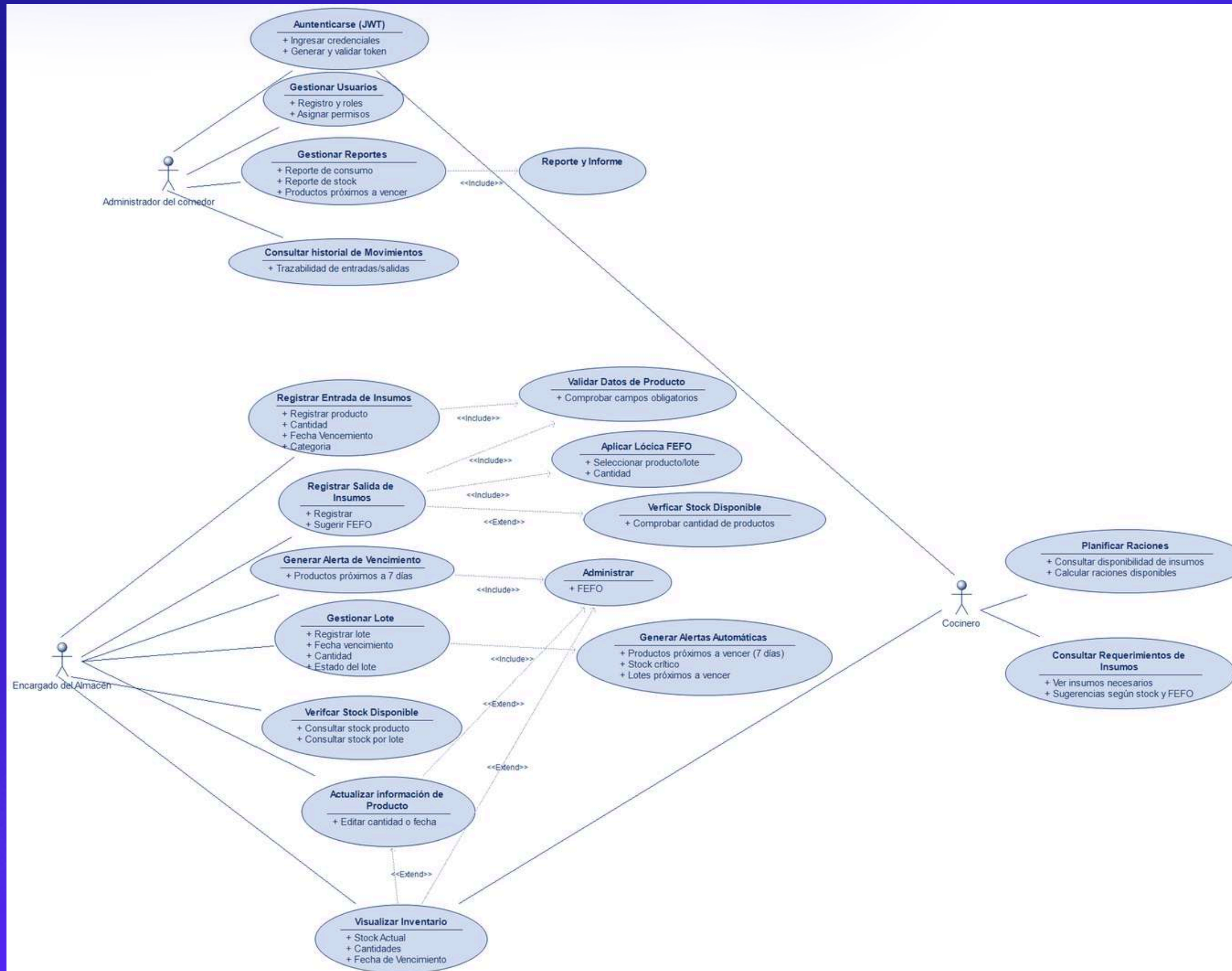


DIAGRAMA- MODELO DE PROCESOS

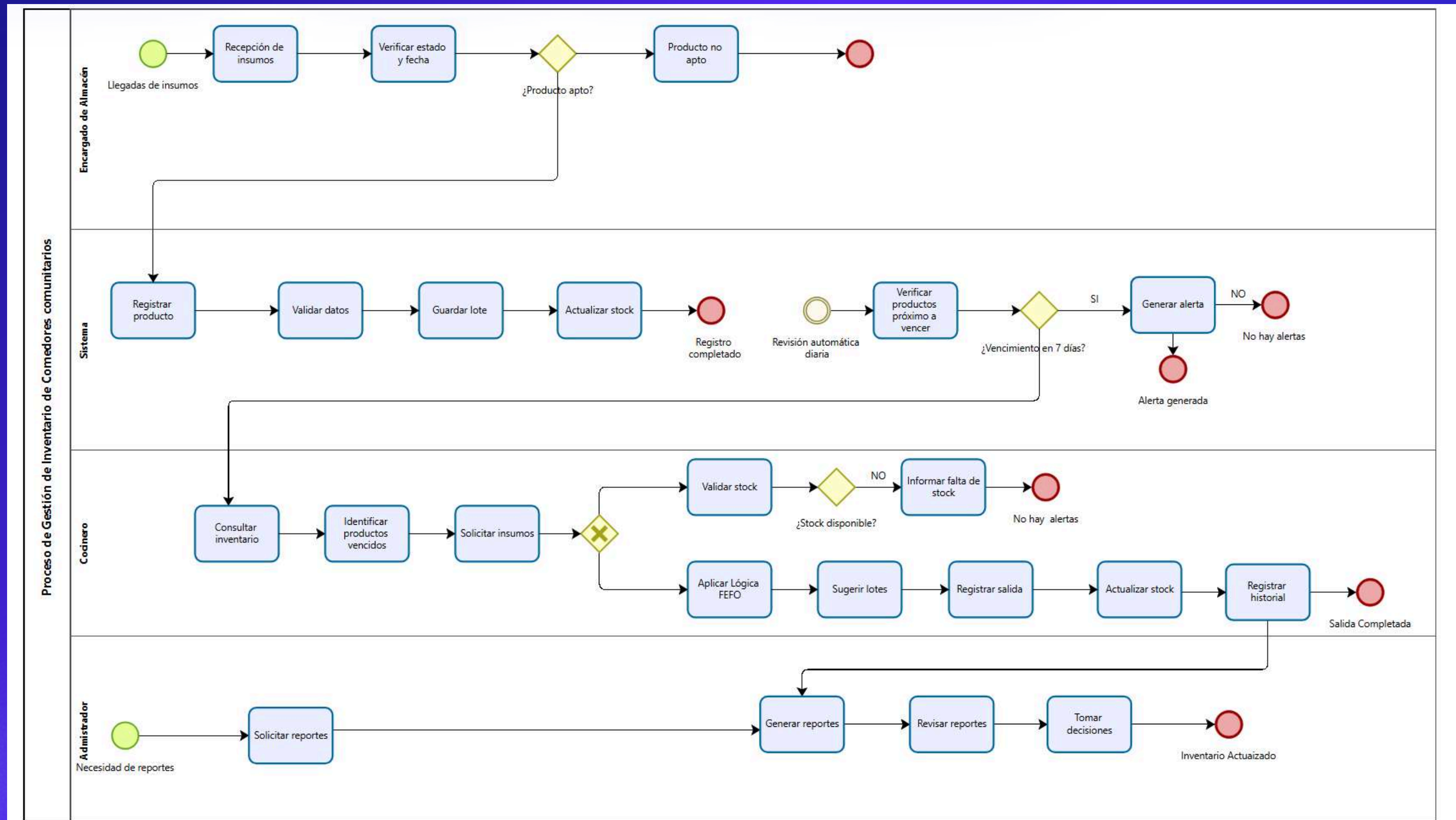


DIAGRAMA- ENTIDADA RELACIÓN



DIAGRAMA DE ENTIDAD-RELACIÓN (DER) CONCEPTUAL - GESTIÓN DE INVENTARIO DE ALIMENTOS

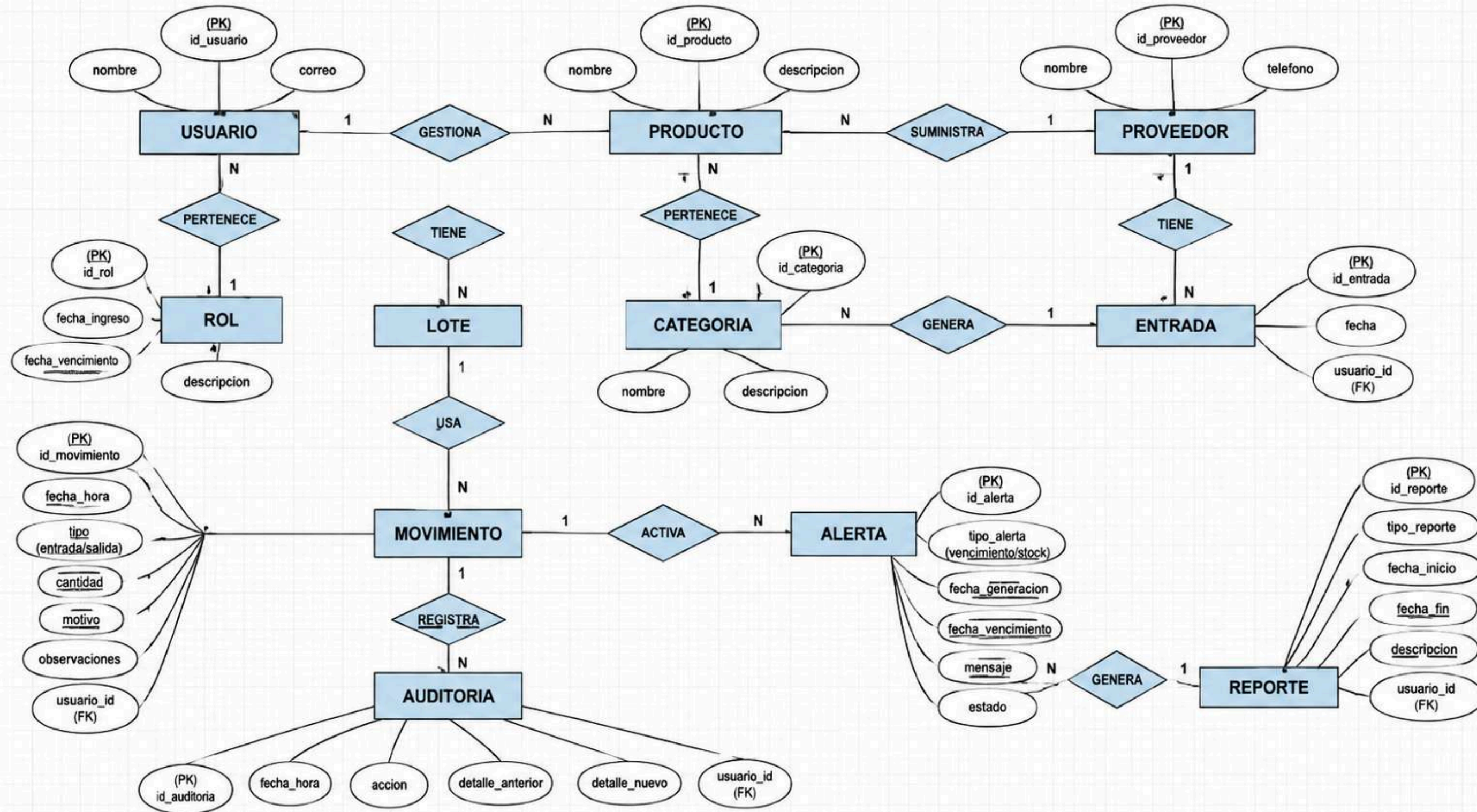
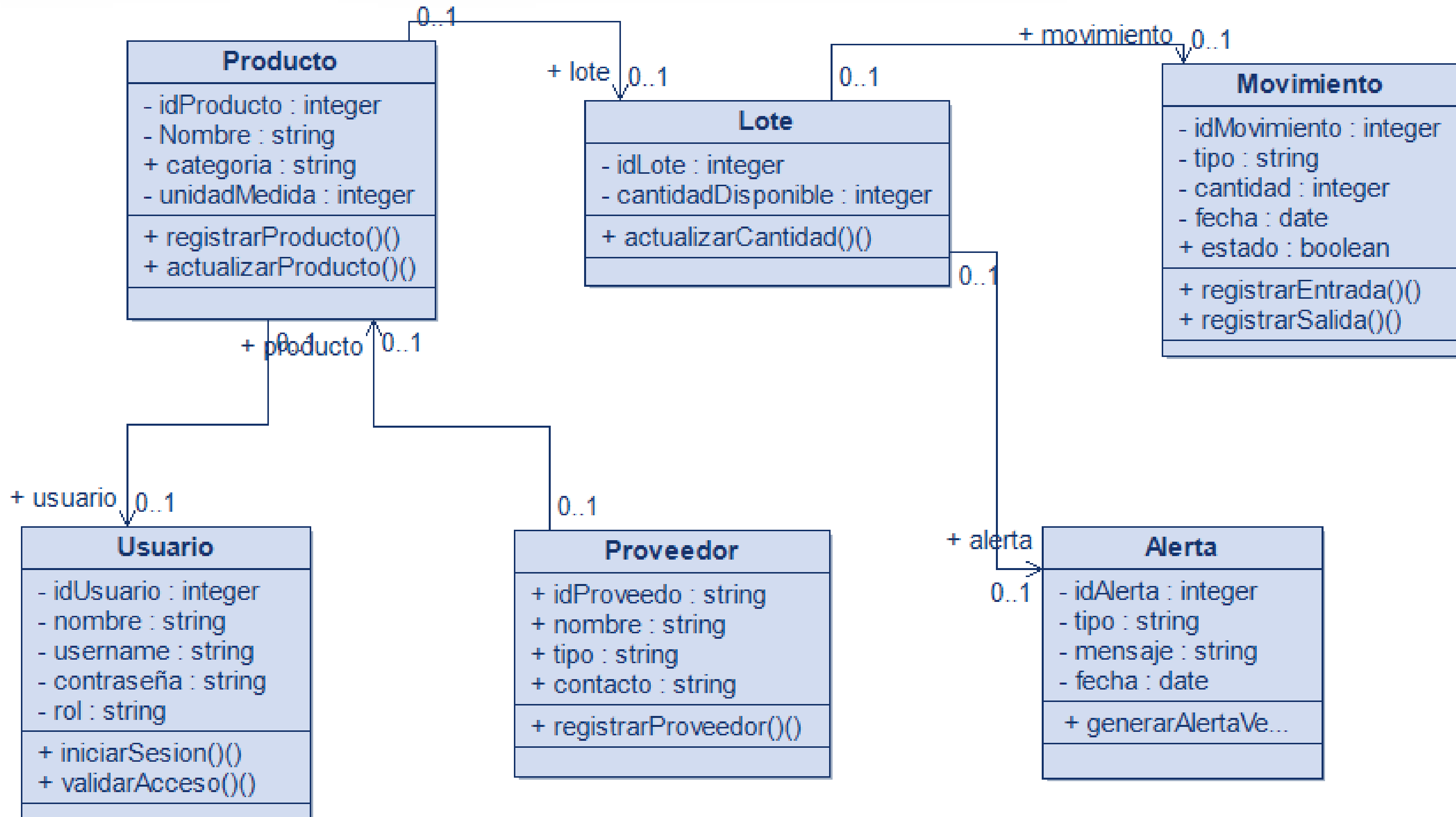
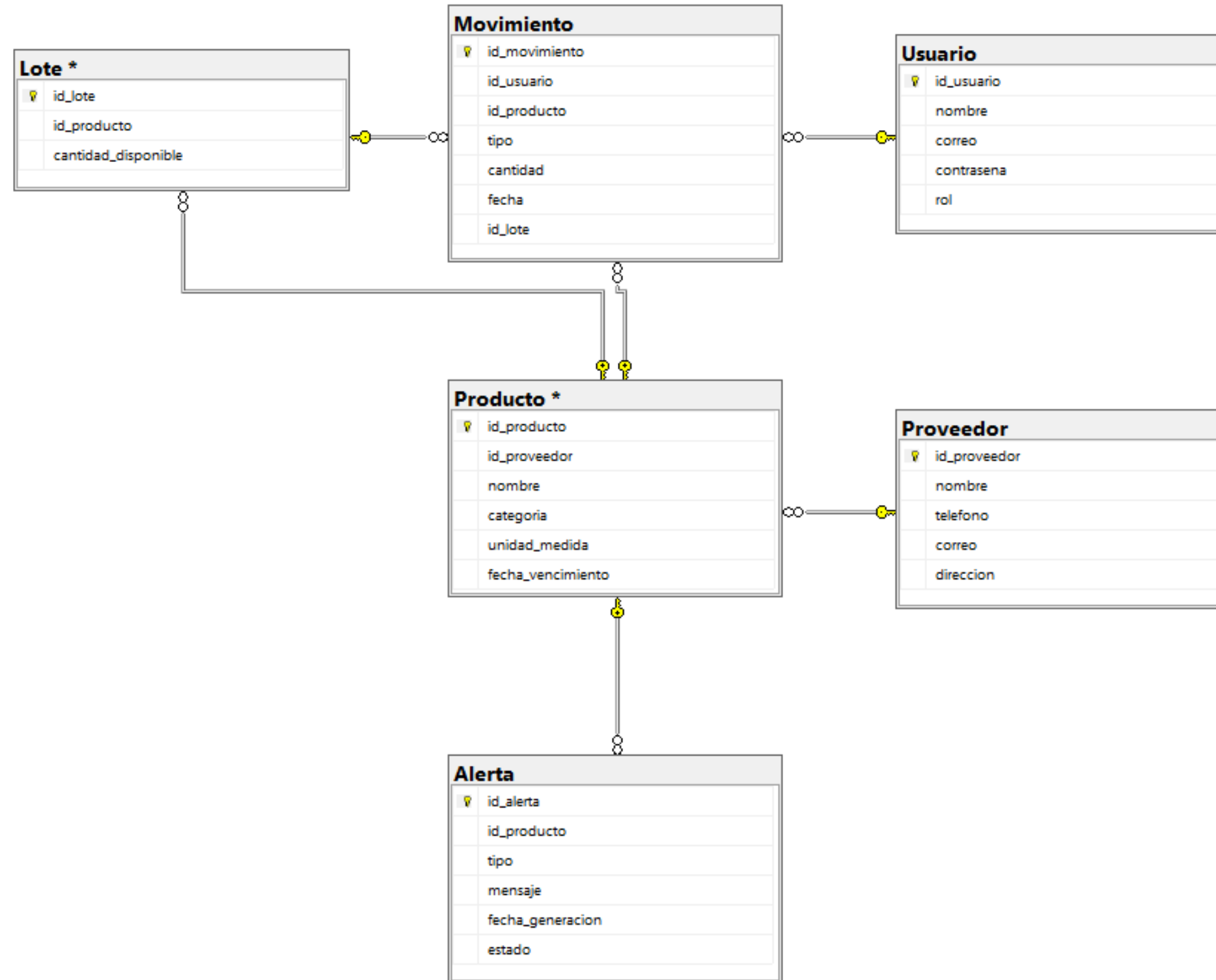


DIAGRAMA- MODELO ESTRUCTURAL



DISEÑO DETALLADO DE LA BASE DE DATOS



ARQUITECTURA SELECCIONADA: MODELO VISTA CONTROLADOR (MVC)



La arquitectura Modelo Vista Controlador (MVC) organiza el sistema en tres componentes principales: Modelo, Vista y Controlador. Esta separación permite distribuir las responsabilidades de manera clara, facilitando el desarrollo, mantenimiento y escalabilidad de la aplicación.

- **Modelo (Model):** administra los datos y la lógica de negocio del sistema, como productos, lotes, movimientos y usuarios.
- **Vista (View):** representa las interfaces que utilizan los usuarios para interactuar con el sistema.
- **Controlador (Controller):** recibe las solicitudes de los usuarios, procesa la información y coordina la comunicación entre la Vista y el Modelo.

ittis diam.

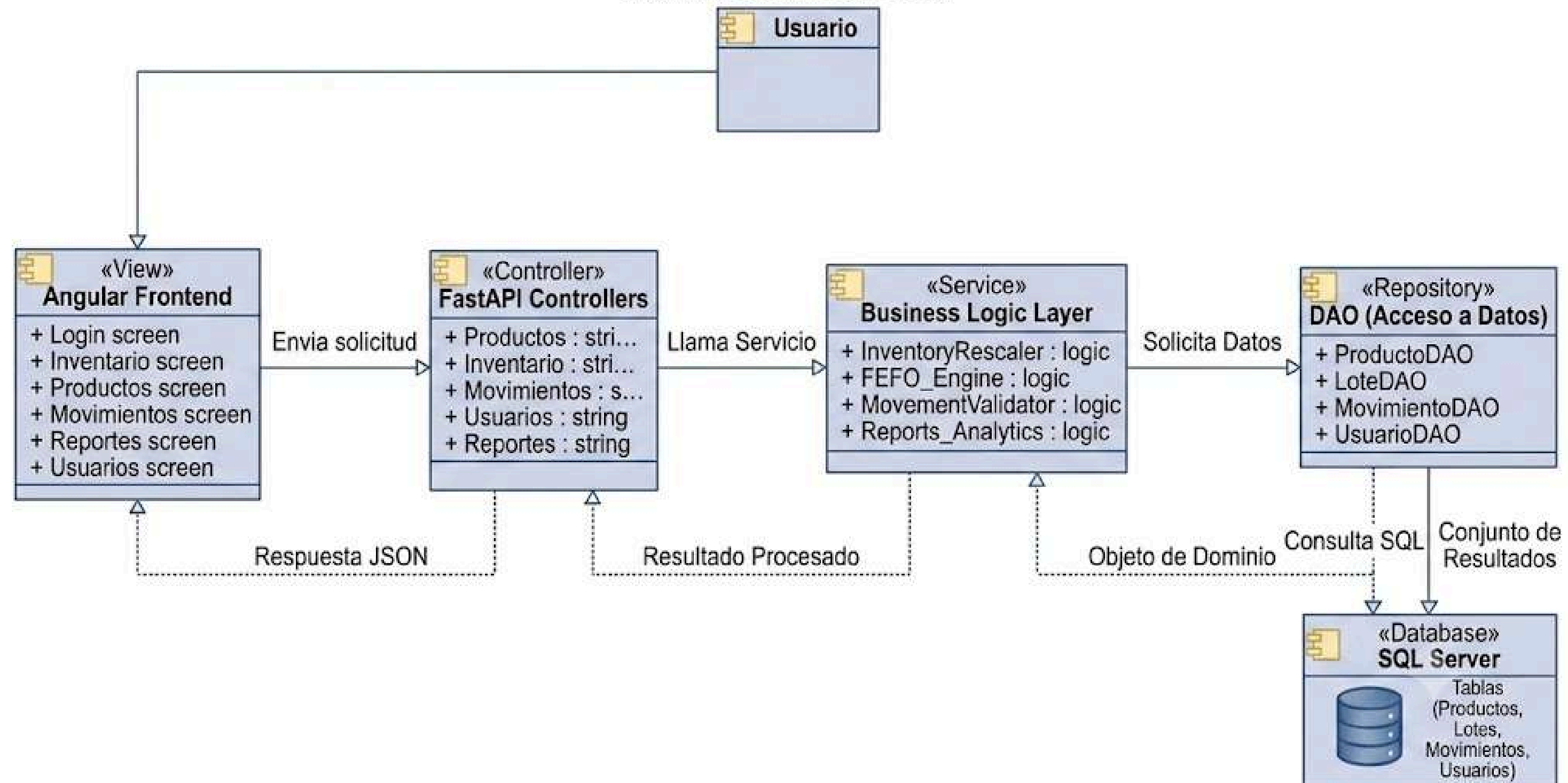


MVC + SERVICE LAYER + DAO



La arquitectura MVC + Service Layer + DAO permite separar las responsabilidades del sistema en múltiples capas, facilitando la mantenibilidad, escalabilidad y reutilización del código. La capa MVC gestiona la interacción con el usuario, la capa Service concentra la lógica de negocio y la capa DAO administra el acceso a la base de datos.

Arquitectura Multicapa con Capa de Servicio y DAO (Angular | FastAPI | SQL Server)



PATRONES- SINGLETON

- En el contexto de un sistema transaccional como la gestión de inventarios, donde múltiples módulos interactúan de forma concurrente con la base de datos, la gestión eficiente de las conexiones es crítica para el rendimiento y la estabilidad. El patrón Singleton se aplica a la clase **ConexionBD** para garantizar que exista una única instancia de conexión durante toda la ejecución del sistema. Esta centralización evita la creación innecesaria de múltiples conexiones simultáneas, optimizando el uso de recursos del servidor y mejorando el tiempo de respuesta.
- **Lotes:** Al administrar múltiples lotes por producto, se realizan consultas y actualizaciones frecuentes (entrada de lotes, salida FEFO, stock crítico). El Singleton centraliza y optimiza el uso de la conexión para estas operaciones concurrentes, garantizando que el acceso a datos sea eficiente incluso bajo carga.
- **Movimientos:** Centralizar la conexión es crucial para garantizar que las transacciones complejas, como el registro de un movimiento que implica actualización de stock y selección de lote FEFO, se realicen de forma atómica y segura sobre la misma conexión gestionada por el Singleton.
- **Alertas:** Las alertas automáticas por vencimiento o stock crítico (generadas por un servicio periódico o asíncrono) requieren acceso eficiente a los datos para consultar estados y fechas. El Singleton permite este acceso sin sobrecargar el servidor de conexiones.

PATRONES- DAO

- El patrón DAO se implementa para separar la lógica de acceso a datos de la lógica de negocio de la aplicación, proporcionando una interfaz abstracta y uniforme para interactuar con la base de datos SQL Server. Cada entidad dispone de una clase especializada (*DAO) y una implementación (*DAOImpl) que encapsulan las operaciones CRUD y las consultas específicas del módulo, ocultando los detalles de la persistencia y facilitando la mantenibilidad, escalabilidad y pruebas unitarias.
- **Lotes:** LoteDAO encapsula la lógica para crear nuevos lotes (fecha ingreso, vencimiento), consultar lotes disponibles para un producto, y actualizar las cantidades de lotes específicos. La lógica FEFO (seleccionar el lote con vencimiento más próximo) interactuará con métodos avanzados de LoteDAO para recuperar lotes ordenados por fecha.
- **Movimientos:** MovimientoDAO encapsula la lógica para registrar cada transacción de entrada y salida, asegurando la integridad de los datos obligatorios (usuario responsable, lote utilizado, fecha, motivo). También proporciona métodos para consultar el historial de movimientos de un producto o en un período.
- **Alertas:** AlertaDAO gestiona las alertas generadas por vencimiento y stock crítico. Permite crear nuevas alertas y consultar alertas activas, pendientes o atendidas. Interactúa con LoteDAO para verificar fechas y ProductoDAO para stock.
- **Usuarios:** UsuarioDAO encapsula la lógica para el registro y autenticación de usuarios. Proporciona métodos para verificar credenciales y recuperar roles de usuario, facilitando la implementación de la autenticación JWT y el control de acceso basado en roles (RBAC).
- **Reportes:** ReporteDAO encapsula la lógica para generar reportes complejos. En lugar de operaciones CRUD directas, proporciona métodos que ejecutan consultas agregadas y calculan métricas como desperdicio evitado e impacto ODS 2, abstrayendo las consultas SQL complejas de la lógica de presentación.

MATRIZ DE TRAZABILIDAD

Req	Descripción	Caso de uso	Clase o módulo	Pantalla del prototipo	Cumple / Parcial / No cumple
RF01	Registrar lotes de productos con cantidad y fecha de vencimiento.	Registrar Entrada de Insumos	Controlador de productos / Entidad: Producto	Pantalla 4: Agregar Nuevo Producto	Cumple. Se incorporó la entidad Lote al modelo físico y a la base de datos, permitiendo registrar múltiples ingresos de un mismo producto y facilitando la gestión del inventario.
RF04	Registrar salida descontando stock y aplicando lógica FEFO.	Registrar Salida de Insumos	Controlador de movimientos / Entidad: Movimiento	Pantalla 6: Gestión de Movimientos	Cumple. La tabla Movimiento se relaciona con la entidad Lote , permitiendo identificar el lote utilizado en cada movimiento y aplicar la lógica FEFO.
RF05	Mostrar estado actual del inventario, cantidades y vencimientos.	Visualizar inventario	Controlador de inventario / Vista: Inventario	Pantallas 2 y 3: Dashboard e Inventario	Cumple. Interfaz clara y coherente con las métricas solicitadas.

MATRIZ DE TRAZABILIDAD

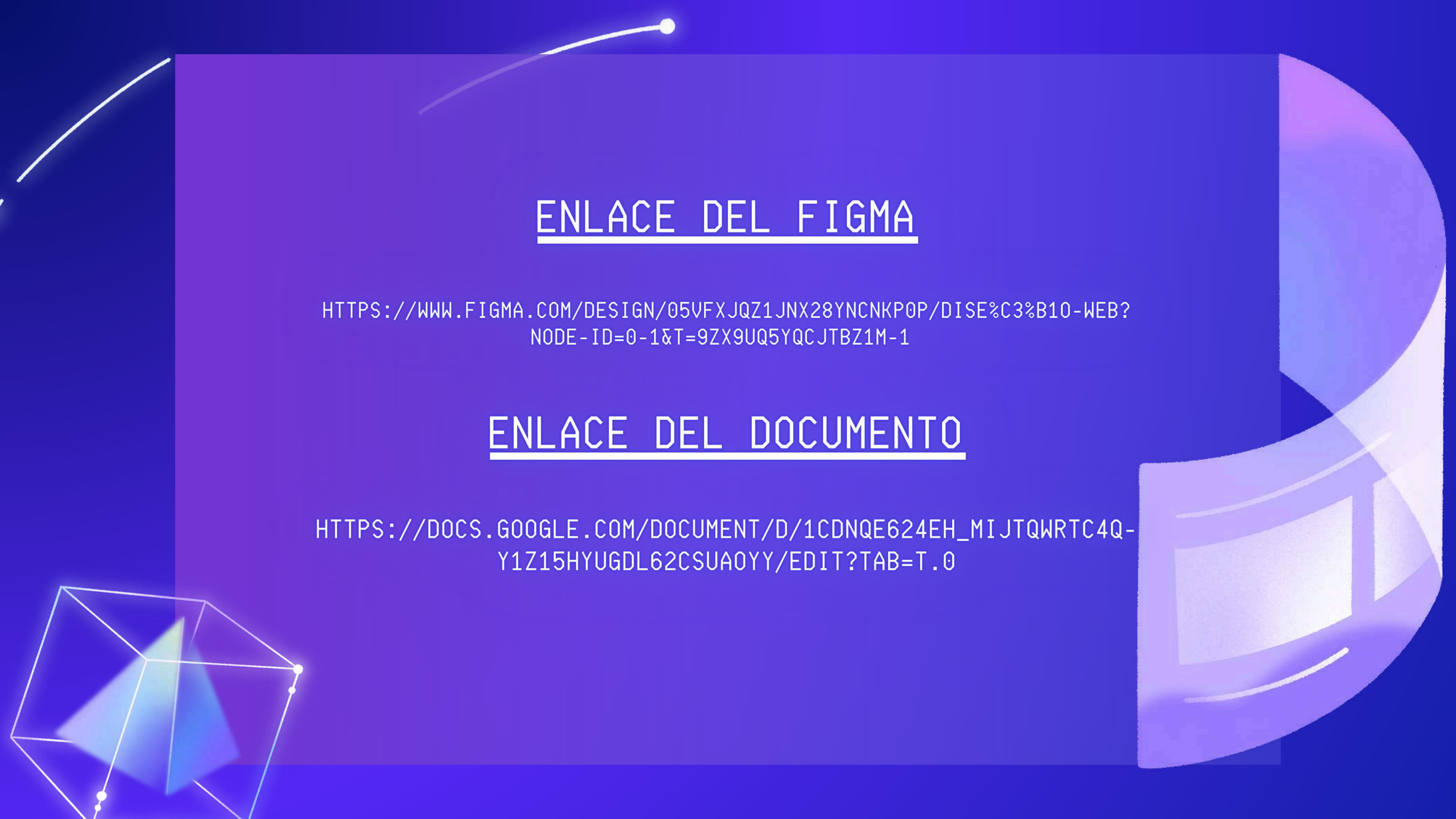
RF06	Generar alertas automáticas cuando faltan 7 días para el vencimiento.	Generar Alerta de Vencimiento	Entidad: Alerta	Pantalla 7: Gestión de Alertas	Cumple. El sistema genera alertas cuando faltan siete días para el vencimiento del producto, manteniendo coherencia entre requerimientos, prototipos y diseño.
RF09	Gestión y autenticación de usuarios con distintos roles.	Gestionar Usuarios / Autenticarse	Controlador de usuarios / Entidad: Usuario	Pantalla 1: Login	Parcial. No hay pantallas para administrar roles (RBAC) y existe ambigüedad en la definición de perfiles.
RF10	Mostrar métricas ODS 2 (desperdicio evitado, alimentos aprovechados).	Gestionar Reportes	Módulo Reportes / Entidad Reporte	Pantalla 8: Informes y Estadísticas	Parcial El prototipo muestra datos que el modelo de base de datos actual
RNF01	Interfaz simple: registro en máximo 30 segundos y 3 pasos.	N/A	Frontend (HTML/CSS/JS)	Todas las pantallas	Cumple. Cumple con los principios heurísticos de consistencia y simplicidad (Nielsen, 1994).
RNF10	Integridad: Movimientos inmutables con trazabilidad histórica.	Consultar historial de Movimientos	Base de Datos (SQL Server)	Pantalla 6: Gestión de Movimientos	Parcial. Los movimientos quedan registrados en la base de datos y mantienen su trazabilidad mediante la relación con la entidad Lote. Como mejora futura, se implementarán mecanismos de auditoría para impedir la modificación o eliminación de registros históricos.

EVALUACIÓN DEL DISEÑO DEL SISTEMA

- El diseño presenta fortalezas en la separación de responsabilidades gracias a la arquitectura MVC (Buschmann et al., 1996); sin embargo, se identifican brechas críticas de coherencia:
- Incorporación de la entidad Lote: Se añadió la entidad Lote al modelo físico y a la base de datos, permitiendo registrar diferentes cantidades de un mismo producto y fortaleciendo la trazabilidad de los movimientos de inventario.
- Inconsistencia Arquitectónica: La arquitectura carece de la representación de componentes que ejecuten reglas de negocio críticas, tales como la automatización de alertas o el control de acceso basado en roles. La justificación de los patrones DAO (Data Access Object) y Singleton es genérica y no aclara su instancia.
- Desalineación de Datos y Prototipos: El prototipo exhibe funcionalidades de "Desperdicio Evitado" y "Mermas"; no obstante, el modelo lógico y físico no posee tablas ni atributos capaces de almacenar o calcular dichos indicadores orientados a la sostenibilidad (Naciones Unidas, 2015).

CONCLUSIÓN

En conclusión, el proyecto desarrollado permitió diseñar un sistema de gestión de inventario alimentario alineado con el ODS 2, orientado a reducir el desperdicio de alimentos mediante la aplicación de la lógica FEFO, el control de lotes y la generación de alertas de vencimiento. Asimismo, la incorporación de una arquitectura multicapa basada en MVC + Service Layer + DAO permitió mejorar la organización, mantenibilidad y escalabilidad del sistema, garantizando una adecuada separación de responsabilidades. Finalmente, la integración de módulos de auditoría, reportes, roles y trazabilidad fortaleció la consistencia del sistema, permitiendo una solución más completa, segura y acorde a los principios de análisis y diseño de software.



ENLACE DEL FIGMA

[HTTPS://WWW.FIGMA.COM/DESIGN/05VFXJQZ1JNX28YNCNKPOP/DISE%C3%B1O-WEB?
NODE-ID=0-1&T=9ZX9UQ5YQCJTbZ1M-1](https://www.figma.com/design/05VFXJQZ1JNX28YNCNKPOP/DISE%C3%B1O-WEB?node-id=0-1&t=9ZX9UQ5YQCJTbZ1M-1)

ENLACE DEL DOCUMENTO

[HTTPS://DOCS.GOOGLE.COM/DOCUMENT/D/1CDNQE624EH_MIJTQWRTC4Q-
Y1Z15HYUGDL62CSUA0YY/EDIT?TAB=T.0](https://docs.google.com/document/d/1CDNQE624EH_MIJTQWRTC4Q-Y1Z15HYUGDL62CSUA0YY/edit?tab=T.0)

MUCHAS GRACIAS

POR VER ESTA PRESENTACIÓN

